

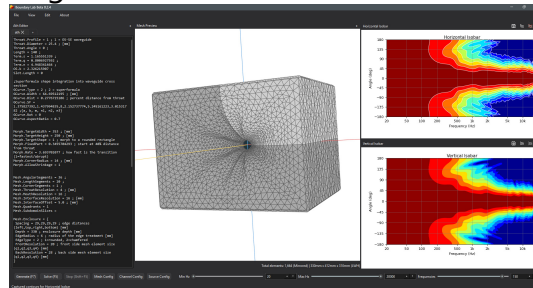
# Boundary Lab Guide

This guide was generated automatically from markdown files inside `/docs/`.

## Main Window

The main window is divided into 3 panes in the default view:

- Ath script editor on the left
- 3D viewport in the middle
- Directivity plots on the right



Each panel can be resized, moved, or dragged out of the main window. If a panel is closed, it can be reopened from the View menu at the top of the window.

The command strip along the bottom of the window contains geometry generation, solve controls, mesh/source/channel configuration, and frequency range settings.

## Ath Script Editor

The Ath script editor contains a text editor for defining Ath shapes. Existing Ath `.cfg` files can be imported/exported into this editor using `File > Import/Export .cfg`.

```
2way X +
Throat.Profile = 1 ; 1 = OS-SE waveguide
Throat.Diameter = 40 ; [mm]
Throat.Angle = 45 ; [deg]
Length = 30 ;
Term.s = 1 ;
Term.q = 0.63 ;
Term.n = 4 ;
OS.k = 3.3 ;
Slot.Length = 0
Coverage.Angle = 36 + 17*cos(p)^2 ; [deg]

;Superformula shape integration into waveguide cross section
GCurve.Type = 2 ; 2 = superformula
GCurve.Width = 90 ; [mm]
GCurve.Dist = .5 ; percent distance from throat
GCurve.SF = 1.3,1,4,5,5,5
GCurve.Rot = 0
GCurve.AspectRatio = 1

;source definition of a dome tweeter
Source.Contours = {
  zoff 0
  point p1 6.8 0 1.5
  point p2 0 12.8 2
  point p3 1.7 16.1 2
  point p4 0 19.2 2
  point p5 0 20 2
  cpoint c1 -0.5 0
  cpoint c2 -2 16
  arc p1 c1 p2 1.0
  arc p2 c2 p3 0.75
  arc p3 c2 p4 0.25
  line p4 W0 0
}
Source.Velocity = 2

Morph.TargetWidth = 220 ; [mm]
Morph.TargetHeight = 150 ; [mm]
Morph.TargetShape = 2 ; morph to a rounded rectangle
Morph.FixedPart = 0 ; start at 40% distance from throat
Morph.Rate = 5 ; how fast is the transition (1=fastest/abrupt)
Morph.CornerRadius = 10 ; [mm]
Morph.AllowShrinkage = 1

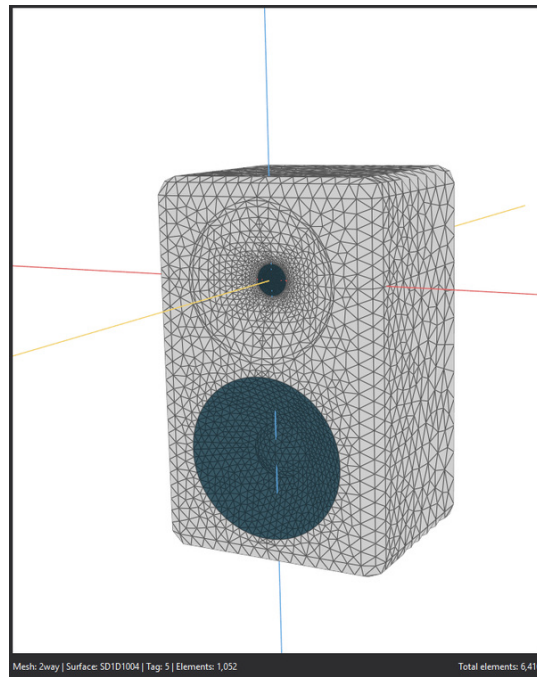
Mesh.AngularSegments = 64 ;
Mesh.LengthSegments = 10 ;
Mesh.CornerSegments = 4 ;
Mesh.ThroatResolution = 3 ; [mm]
Mesh.MouthResolution = 16 ;
Mesh.Quadrants = 14
Mesh.Subdomainslices =

Mesh.Enclosure = {
  Snacing = 50 25 50 250 ; edge distances (left top right bottom) [mm]
```

Ath scripts can be added, removed, and renamed using the tab controls at the top of the editor pane. To rename a script, double click its tab. Multi-script workflows can be useful for complex multiway designs (see /examples/MultiAth+Mesh\_3WayIntegrated), or workflows where you might be comparing outputs on the same script with different values by copy/pasting it into multiple script tabs and enabling/disabling them in the `Mesh Config` window.

## 3D Viewport

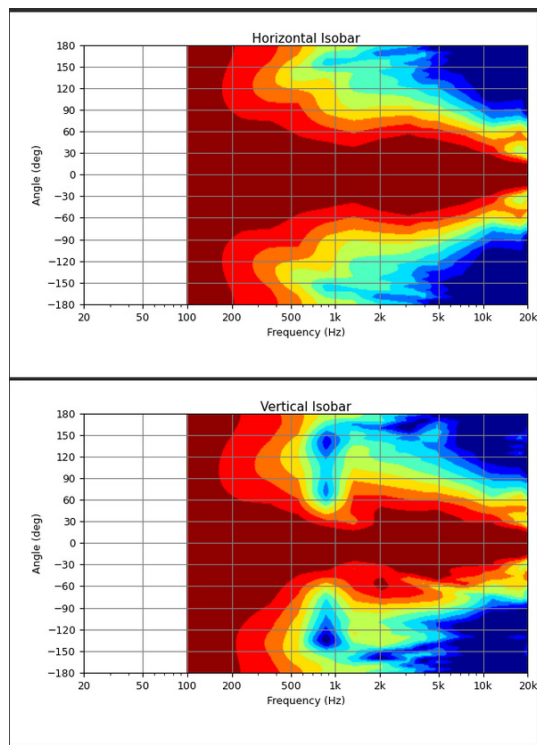
The 3d viewport contains all active mesh files in the project. Surfaces are color coded as blue for driven and grey for rigid. Hovering over the meshes displays the mesh name, surface name, surface index, and number of elements for that surface group. The bottom right of the 3D viewport displays the total element count for all active meshes.



The viewport displays a yellow line for the on-axis vector, a blue line for the vertical axis, and a red line for the horizontal axis emanating from the origin. All directivity calculations are performed relative to the origin, with the on-axis direction facing along the +z axis. Use the mesh config window to move the active mesh files in 3D space so that the appropriate location for the origin is used.

## Plot Views

Plots are displayed when selected from the `view` menu. Results are streamed in realtime to the plots as solves are ran.



# Menu Bar

## File Menu

- `New Project`: clears the editor, mesh setup, source setup, preview, and solved data.
- `Save Project`: saves the current project to the active `.blab.json` file.
- `Save Project As`: chooses a new `.blab.json` path.
- `Load Project`: loads editor text, mesh config, and source config.
- `Import .cfg`: imports only Ath config text into the editor.
- `Export .cfg`: exports only the editor contents.
- `Export Plot`: exports generated plot panels as PNG files.
- `Export Polar Data`: exports solved horizontal and vertical polar text files. Channel-basis solves export frequency, normalized SPL, and relative phase.

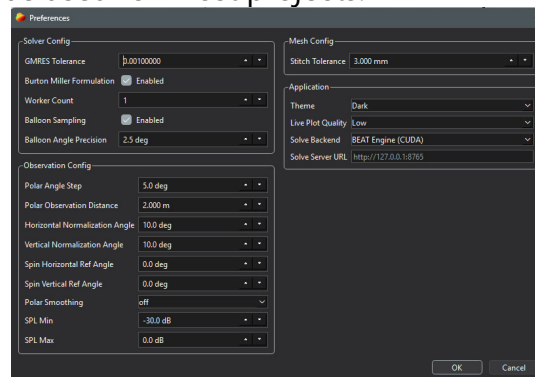
Project files do not store solved results or global preferences.

## View Menu

The View menu enables/disables various plots that are displayed in the plot viewer pane of the main window. It also opens the balloon plot viewer in a new window. The balloon plot viewer can only be opened once there is a completed or stopped solve.

## Edit/Preferences Menu

The preferences menu contains various application-level settings for Boundary Lab. The default values for these settings can be used for most projects.



## Solver Config

- `GMRES Tolerance`: sets the threshold for how accurate the BEM solution needs to be at each frequency being solved. Lower values (moving the 1 to the right) increase solve times but may produce more accurate results. High values (moving the 1 to the left) decrease solve accuracy but also decrease solve times.
- `Burton Miller Formulation`: enable/disable the Burton-Miller formulation to prevent the exterior Helmholtz boundary integral equation from becoming unreliable at

certain frequencies due to fictitious cavity resonances. Not always required but turning this feature on can reduce polar irregularities with certain meshes- especially ones having enclosed volumes. Turning this feature off can typically decrease solve times by 30-40%.

- `Worker Count`: Experimental - allocate CPU worker threads to the solver. The default value is 1, but changing this may either increase or decrease solve times depending on your hardware and mesh complexity.
- `Balloon Sampling`: Enable or disable spherical sampling needed to generate balloon plots. This feature generates a 2 meter diameter grid of sampling points around the origin according to a Fibonacci spherical sampling sequence to approximate equal point spacing. Enabling/disabling this feature typically has a minimal (<3%) impact on performance.
- `Balloon Angle Precision`: Drives the number of points generated for the spherical balloon plot sampler. More points produce more detailed balloon plots, but slightly increase run times. The default value of 2.5 degrees produces 6,500 points in a sphere around the origin to sample from.

## Observation Config

- `Polar Angle Step`: The number of degrees between each observation point that generates the horizontal/vertical polar plots. A step angle of 5 degrees evaluates 72 points across a 360 degree arc on each axis. If you intend to view the Spinorama-style plot, ensure this value is set to max 10 degrees. Changing this value has extremely little impact on solve times (<1%).
- `Polar Observation Distance`: Set the distance from the origin for the field point sampling positions. Default is 2 meters.
- `Normalization Angles`: Set the polar normalization angles used to offset SPL values in polar directivity plots and balloon plots.
- `Spin Horizontal/Vertical Ref Angle`: Set the reference axis angles used by the spinorama-style reference-axis and listening-window curves. These angles do not renormalize the polar data used for early reflections or sound power.
- `Polar Smoothing`: Set the smoothing scale for polar directivity plots, spinorama-style plots, and balloon plots.
- `SPL Min`: Set the minimum clipped SPL value for generating polar directivity & balloon plots. Simulated SPL values below this are clipped to this value as the floor.
- `SPL Max`: Set the maximum clipped SPL values for generating polar directivity & balloon plots. Simulated SPL values above this are clipped to this value as a ceiling.

## Mesh Config

- `Stitch Tolerance`: Set the search distance for nearby open edges when combining multiple mesh files in the same project.

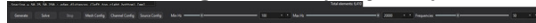
- **Symmetry:** Enable half or quarter symmetry that mirrors all active meshes along the X or X/Y axes for significantly faster solving. Supported by the BEAT Engine CUDA and BEAT Engine CPU solvers. Mirrored mesh elements are shaded darker in the 3d viewport.

## Application

- **Theme:** Boundary Lab visual UI theme.
- **Solve Backend:** Set the solver backend to either use a built-in application solver, or a server-based backend.
  1. **Server** - Use whatever solver is configured on the remote server via the HTTP streaming API.
  2. **BEAT Engine (CUDA)** - Use the Julia-based CUDA solver. BEAT Engine is short for Boundary Element Acoustic Toolkit Engine. Requires Julia and an NVIDIA GPU with a working CUDA-capable driver.
  3. **BEAT Engine (CPU)** - Use the Julia-based CPU solver path. Uses the local CPU BLAS/LAPACK stack and supports BEAT Engine X/XY symmetry acceleration.
  4. **Bempp (OpenCL CPU)** - Use the bempp-cl OpenCL solver backend. Requires an OpenCL runtime to be installed.
- **Solve Server URL:** The address and port of the server if using a server-based solver backend.

## Command Strip

The command strip is located along the bottom of the main window and includes controls to generate Ath meshes, run solves, and configure the project parameters.



### Generate

Click **Generate** to run `ath/ath.exe` against the editor text. Boundary Lab writes the temporary `.cfg`, lets Ath generate geometry, cleans the generated mesh, and loads it into the preview.

Ath outputs are written under:

`runs/ath_output`

### Solve

Initiates the boundary element method (BEM) solver against the current project as it is represented in the 3d viewport and configurations.

Note - the first time solve after opening Boundary Lab will typically take longer as the tool loads backend libraries and initializes OpenCL caches. Subsequent solves utilize the OpenCL warmed caches for faster runtimes.

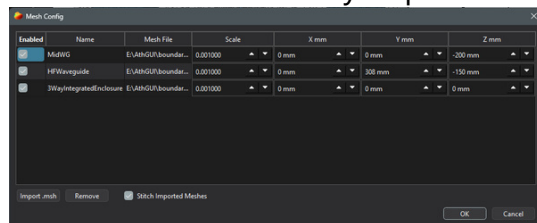
### Solve/Stop

Click **Solve** to start the BEM sweep. Enabled plots update in realtime as each frequency

completes. Click **Stop** to stop after the current in-flight frequency finishes; completed frequencies remain available for plotting/export.

## Mesh Config

**Mesh Config** lists the generated Ath meshes and any imported **.msh** files.



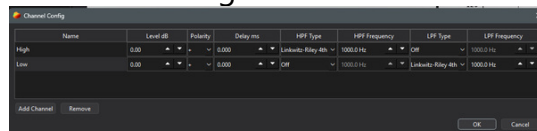
If you've generated any Ath meshes, they will be visible as the top rows in the mesh config window.

Imported **.msh** rows can be enabled/disabled, renamed, removed, scaled, and translated.

When imported meshes are enabled, they are included in the preview and solve. If mesh stitching is enabled in Mesh Config, Boundary Lab can stitch the active generated/imported meshes before solving. The stitching toggle is saved with the project.

## Channel Config

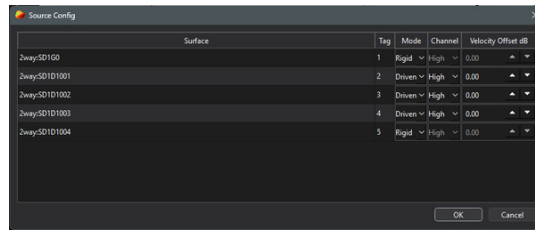
**Channel Config** is used to define the channels present in the loudspeaker. It is only used when solving multiway loudspeaker projects where the effects of source-source interference patterns are important. Changes to the channel config affect all sources allocated to that channel.



- **Name:** Double click to define the channel name
- **Level dB:** apply a level offset to this channel
- **Polarity:** sets the channel polarity
- **Delay ms:** sets the delay for this channel in milliseconds
- **HPF/LPF Type:** Sets acoustic highpass/lowpass filters for this channel. Reference the [Model Assumptions help article](#) for more information about how Boundary Lab applies crossover filters
- **HPF/LPF Frequency:** Set the frequency of the highpass/lowpass filters

## Source Config

**Source Config** lists mesh surface groups discovered from the active meshes. Use it to mark driven radiator surfaces and set their assigned channel and gain offset.



Generally, the gain offset feature should be used to define complex radiating sources for more accurate BEM modelling. For example, splitting a dome tweeter into 3 surface groups (dome, inner surround, outer surround), allows you to taper off the drive level towards the surround edge. Load the /examples/Ath\_2WayBookshelf/ example to see how this can be configured.

Only driven surfaces are excited during the BEM solve. All other surfaces are treated as rigid boundaries. Recall that the 3d viewport displays surface group names when you hover the mouse over the mesh preview.

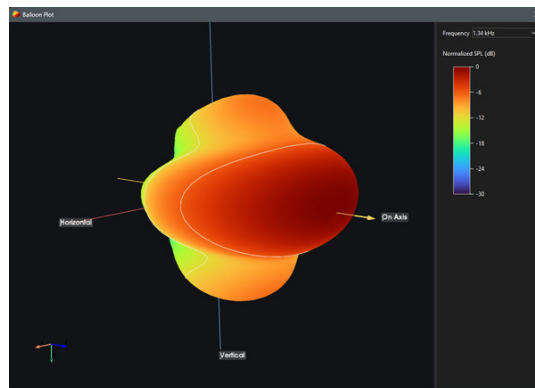
## Frequency Controls

The frequency controls set the minimum, maximum, and number of frequencies that will be solved when clicking "Solved". The solver automatically uses logarithmic spacing according to the min/max/number.

## Balloon Plot

After a solve with spherical sampling enabled, open:

View > Balloon Plot



The viewer comprises:

- rotatable/zoomable 3D directivity balloon
- frequency picker
- SPL color legend
- 6 dB contour lines drawn on the surface of the balloon
- horizontal, vertical, and on-axis guide lines
- rotatable polar protractor with 30 degree angle spokes and 6 dB rings
- radar directivity plot for the current frequency and protractor angle

- isobar slice rendering for the current polar protractor angle

The balloon viewer uses the frequencies that completed before the solve ended.

Use `File > Export Balloon Data` in the balloon plot window to export a fixed-topology balloon dataset for external visualization tools. The export folder contains metadata, shared surface topology, normalized SPL values, normalized radius values, and per-frequency XYZ positions for the balloon surface only. It does not include contour lines, guide axes, protractor geometry, or radar/isobar plots.

## Project Files

Boundary Lab project files use readable JSON and the `.blab.json` extension.

Project files store:

- Ath `.cfg` editor text
- generated Ath mesh enabled state and XYZ offset
- imported mesh rows, including absolute `.msh` paths
- whether imported meshes should be stitched into a single solve mesh
- source configuration by surface name

Project files do not store:

- solved BEM results
- exported plots
- global preferences
- generated Ath output files

## Example Shape

```
{
  "schema_version": 1,
  "ath_config_text": "...",
  "ath_mesh": {
    "name": "ath",
    "source_file": "E:/AthGUI/boundary-lab/runs/ath_output/case/ABEC_FreeS",
    "cleaned_file": null,
    "translation_mm": [0, 0, 0],
    "enabled": true
  },
  "imported_meshes": [],
  "stitch_imported_meshes": false,
  "source_config_by_name": {}
}
```

## Loading Projects

Loading a project updates the editor, mesh config, and source config. It does not automatically run Ath or start a solve.

If the project references imported mesh files, those paths are expected to exist on the local machine. Relative mesh and generated-output paths are resolved from the project file's directory, which keeps bundled samples portable.

## Exporting Plot Images

Users can export any of the rendered plots as .png images.

## Exporting Polar Data

Users can export simulated polar data as individual .txt files per angle sampled for horizontal and vertical axes. Channel-basis solves export three tab-separated columns: frequency in Hz, normalized SPL in dB, and relative phase in degrees. The .txt files can be directly imported into tools such as REW and VituixCAD for external analysis.

# Model Assumptions

Boundary lab is a BEM-only solver application. As a result, it is important to define what it can and cannot accomplish in the context of loudspeaker design. The fundamental use-case of Boundary Lab is to answer the question:

"Given a loudspeaker enclosure and/or waveguide design, what is the resultant normalized directivity produced by the radiating elements contained in the device"

Or:

"Given a multiway loudspeaker enclosure, what is the resultant normalized directivity produced by all radiating elements assuming idealized analog crossover transfer functions, plus user-defined gain, polarity, and delay"

## Source Definition & Driving Model

The application allows for two types of boundaries to be defined:

1. **Rigid**: An infinitely rigid reflective surface with zero normal velocity.
2. **Driven**: A surface with prescribed normal velocity according to an idealized radiator model.

Because each driven source is an idealized model of a perfect radiator, it is important to denote that Boundary Lab does not support electroacoustic modeling of loudspeaker transducers. It does not model diaphragm breakup, suspension compliance, motor force factor, voice-coil impedance, or other transducer behavior. External Lumped Element tools or more advanced FEA tools are preferred for electroacoustic transducer modelling and to simulate internal enclosure, port, suspension, motor, and diaphragm resonances, while Boundary Lab can supplement these tools to perform the directivity analysis.

## Normalized Channel Transfer Function

Despite each source being driven according to an idealized source, the resultant non-normalized reference-axis frequency response is not necessarily flat. This can introduce complexities when modeling crossover slopes since the acoustic targets are not adhered to. To combat this, Boundary Lab computes a per-channel reference-axis magnitude correction from the unit-velocity response of that channel, then applies the user-defined channel gain, polarity, delay, and crossover filters. This correction flattens the isolated channel magnitude target **before** crossover shaping; it does not apply a complex phase inverse and does not guarantee that the final summed multiway response is flat.

## Boundary Integral Equation Model

Boundary Lab solves the exterior acoustic Helmholtz problem in the frequency domain. For each solved frequency, the acoustic pressure  $p$  outside the mesh is assumed to satisfy:

$$\nabla^2 p + k^2 p = 0$$

where:

- $k = \omega / c$
- $\omega = 2 \pi f$
- $c$  is the speed of sound

The mesh surface is treated as the boundary of the exterior acoustic domain. Boundary Lab uses a Neumann boundary condition, meaning the normal pressure gradient is prescribed from the configured surface velocity:

$$q = \frac{\partial p}{\partial n} = i \rho \omega v_n$$

where:

- $q$  is the boundary Neumann data
- $\rho$  is air density
- $v_n$  is the prescribed outward normal surface velocity

Rigid surfaces use  $v_n = 0$ . Driven surfaces use the configured ideal velocity drive, including per-radiator velocity offset and any channel gain, polarity, delay, crossover filtering, and flat-target magnitude correction.

## Discretization

The solver uses Bempp-cl boundary element spaces:

- Boundary pressure  $p$  is represented with continuous linear  $P1$  basis functions.
- Boundary velocity/flux  $q$  is represented with discontinuous constant  $DP0$  basis functions.

At each frequency, Boundary Lab assembles Helmholtz boundary operators on these spaces and solves for the unknown boundary pressure coefficients. The solve is performed with GMRES using the configured solver tolerance.

## Classical Exterior Neumann Form

When Burton-Miller is disabled, Boundary Lab solves the classical exterior Neumann boundary integral equation:

$$(K - \frac{1}{2}I)p = Sq$$

where:

- $S$  is the single-layer boundary operator
- $K$  is the double-layer boundary operator
- $I$  is the identity operator
- $p$  is the unknown boundary pressure
- $q$  is the prescribed Neumann boundary data

This corresponds to the exterior representation used later for field evaluation:

$$p(x) = D[p](x) - S[q](x)$$

where  $D[p]$  is the double-layer potential and  $S[q]$  is the single-layer potential evaluated at observation point  $x$ .

## Burton-Miller Formulation

The classical exterior Helmholtz integral equation can become unreliable at fictitious interior resonance frequencies. Boundary Lab can use a Burton-Miller combined-field formulation to reduce these irregular-frequency artifacts.

When Burton-Miller is enabled, Boundary Lab assembles the double-layer, single-layer, hypersingular, and adjoint double-layer operators:

- $K$ : double-layer boundary operator
- $S$ : single-layer boundary operator
- $W$ : hypersingular boundary operator
- $K'$ : adjoint double-layer boundary operator

The coupling factor is:

$$\alpha = \frac{i}{k}$$

Using Bempp-cl sign conventions, the implemented linear system is:

$$\left(\frac{1}{2}I - K - \alpha(-W_{\text{bempp}})\right)p = \left(-S - \alpha(K' + \frac{1}{2}I)\right)q$$

The code explicitly applies the  $-W_{\text{bempp}}$  term because Bempp-cl's hypersingular operator has the opposite sign from the convention used in the Burton-Miller equation implemented here.

The practical effect is that Boundary Lab combines the pressure equation with its normal derivative equation. This suppresses the fictitious cavity resonances that can appear in the classical exterior Neumann formulation, especially on closed or nearly closed loudspeaker enclosure meshes. Burton-Miller typically increases solve cost because it requires the additional hypersingular and adjoint double-layer operators.

## Field Evaluation and SPL Output

After solving for boundary pressure, Boundary Lab evaluates the acoustic pressure at polar and spherical observation points using:

$$p(x) = D[p](x) - S[q](x)$$

The complex pressure magnitude is converted to SPL with:

$$\text{SPL} = 20\log_{10}\left(\frac{|p(x)|}{20 \times 10^{-6}}\right)$$

The raw solver output is normalized to the horizontal on-axis response. Plotting and export paths can then re-normalize the horizontal and vertical planes to the configured reference angles for directivity visualization.

## BEAT Engine Core

Boundary Lab's BEAT Engine, short for Boundary Element Acoustic Toolkit Engine, is a local direct dense BEM solver used through `src/blab/solvers/beat_engine_backend.py`. The Python side stages mesh assets and request JSON, while `src/blab/solvers/julia_local/solver.jl` owns the numerical solve. `BeatEngineCore.jl` provides shared mesh, quadrature, formulation, symmetry, Burton-Miller, and field-evaluation utilities used by both BEAT Engine CPU and BEAT Engine CUDA.

BEAT Engine solves exterior acoustic radiation from prescribed normal velocity on tagged radiator surfaces. It currently uses single precision (`Float32`) for local solves.

Backend-specific details live in:

- BEAT Engine CPU
- BEAT Engine CUDA

## Solve Pipeline

For each solve request, Julia:

1. Loads one or more Gmsh 2.2 ASCII triangle meshes.
2. Applies per-mesh scale and translation, then combines them into one `BoundaryMesh`.
3. Builds P1 pressure and DP0 velocity spaces.
4. Builds frequency-independent identity/mass matrices.
5. For each frequency, assembles Helmholtz boundary operators.
6. Solves the Burton-Miller Neumann system for boundary pressure.
7. Evaluates SPL at polar and optional spherical observation points.
8. Computes per-radiator acoustic impedance from pressure over driven elements.

Radiators are resolved by both physical tag and mesh id, so duplicate physical tags are allowed across meshes when the radiator specifies its mesh.

## Boundary Integral Formulation

The backend uses the outgoing Helmholtz Green function:

$$G_k(x, y) = \frac{e^{ik\|y-x\|}}{4\pi\|y-x\|}.$$

The assembled operator tuple contains:

- `single_layer`: maps DP0 Neumann data to P1 test functions.
- `double_layer`: maps P1 pressure data to P1 test functions.
- `adjoint_double_layer`: maps DP0 Neumann data to P1 test functions.
- `hypersingular`: maps P1 pressure data to P1 test functions.

The dense Burton-Miller system is formed as:

$$\left(\frac{1}{2}I_{P1, P1} - D + \eta H\right)p = \left(-S - \eta\left(D^* + \frac{1}{2}I_{P1, DP0}\right)\right)q,$$

where:

$$\eta = \frac{i}{k}.$$

Here (p) is the solved boundary pressure and (q) is the Neumann/radiator drive vector. Radiator normal velocity is converted to Neumann data with:

$$q = i\rho\omega v_n.$$

For a test triangle ( $T_i$ ), trial triangle ( $T_j$ ), test basis function ( $\phi_a$ ), and trial basis function ( $\psi_b$ ), a typical single-layer block entry is:

$$S_{ab}^{ij} = \int_{T_i} \int_{T_j} \phi_a(x) G_k(x, y) \psi_b(y) dS_y dS_x.$$

The double-layer and adjoint double-layer use normal derivatives of ( $G_k$ ):

$$D_{ab}^{ij} = \int_{T_i} \int_{T_j} \phi_a(x) \frac{\partial G_k(x, y)}{\partial n_y} \psi_b(y) dS_y dS_x,$$

$$(D^*)_{ab}^{ij} = \int_{T_i} \int_{T_j} \phi_a(x) \frac{\partial G_k(x, y)}{\partial n_x} \psi_b(y) dS_y dS_x.$$

The hypersingular block is assembled with the surface-curl weak form:

$$H_{ab}^{ij} = \int_{T_i} \int_{T_j} G_k(x, y) [\text{curl}_\Gamma \phi_a(x) \cdot \text{curl}_\Gamma \psi_b(y) - k^2 \phi_a(x) \psi_b(y) n_x \cdot n_y] dS_y dS_x.$$

BEAT Engine stores all four dense matrices explicitly. This makes direct dense solves straightforward, but memory usage scales quadratically with the P1/DP0 unknown counts.

## Singular And Regular Pairs

Operator assembly splits triangle pairs into regular pairs and adjacent/coincident singular pairs. Regular pairs are integrated directly with triangle quadrature. Adjacent, edge-sharing, vertex-sharing, and coincident pairs are handled with Duffy correction rules.

The singular path builds a frequency-independent correction cache for the mesh and singular quadrature order. The cache stores adjacent/coincident element pairs, orientation-remapped Duffy rules, surface curls, and pair geometry scalars once, then reuses them across the frequency loop. Per-frequency work still evaluates the Helmholtz kernels because they depend on ( $k$ ).

The image-singular path for symmetry computes compact `singular - regular` correction blocks before adding them to the dense operators. CUDA scatters those blocks through GPU correction buffers with atomics; CPU applies the same correction logic directly on host matrices.

## Symmetry Mode

BEAT Engine supports `off`, `x`, and `xy` symmetry modes in both the BEAT CUDA and BEAT CPU backends. The application disables the symmetry control for backends that do not advertise symmetry support.

The application passes a reduced-domain mesh plus symmetry metadata. BEAT Engine does not infer or match mirrored element orbits. Instead, it assumes the global origin is the symmetry origin and validates that the provided mesh lies in the positive fundamental domain:

- `x`: all mesh vertices must be on or positive of the global  $X=0$  plane.
- `xy`: all mesh vertices must be on or positive of both the global  $X=0$  and  $Y=0$  planes.

For each enabled mirror plane, the backend builds image transforms. Points and normals are reflected as ordinary vectors. Surface curls in the hypersingular weak form are reflected as pseudovectors, using  $\det(R) * R * \text{curl}$  for each mirror transform. For `x` symmetry, the reduced operator includes the identity domain plus the  $X$ -reflected image. For `xy` symmetry, it includes the identity domain plus  $X$ ,  $Y$ , and  $XY$  images.

The reduced Galerkin system remains defined on the reduced mesh's P1 pressure space and DP0 Neumann space. Regular operator assembly handles symmetry by adding reflected trial/source image contributions into the reduced matrices. This keeps the solved pressure vector on the reduced P1 dofs while accounting for the missing physical images.

P1 test rows receive symmetry orbit weights for vertices on symmetry planes. These weights are applied consistently to the Helmholtz operators and to the Burton-Miller identity/mass matrices. This is required for seam vertices on  $X=0$  and  $Y=0$  to match the corresponding full expanded system.

Singular handling has two parts:

- Identity-domain adjacent, edge-sharing, vertex-sharing, and coincident pairs use the normal Duffy correction cache.
- Reflected image pairs that become coincident, edge-adjacent, or vertex-adjacent across a symmetry plane use an image-singular correction cache.

Field evaluation is symmetry-aware by materializing mirrored quadrature sources in the field-evaluation cache. The existing field kernels are then reused unchanged: source points and normals are reflected for each symmetry transform, while source faces, source elements, quadrature weights, and P1 basis values continue to reference the reduced mesh.

Radiator impedance is computed from reduced-domain pressure and then scaled by the symmetry reduction factor: 2 for  $x$ , 4 for  $xy$ . This reports force over the full physical radiator image set while keeping the solve unknowns reduced.

## Field Evaluation

After boundary pressure is solved, the backend evaluates the potential at observation points:

$$u(x) = \int_{\Gamma} \frac{\partial G_k(x, y)}{\partial n_y} p(y) - G_k(x, y) q(y) dS_y.$$

The implementation precomputes a field-evaluation cache containing quadrature source points, normals, weights, source faces, source elements, and P1 basis values.

The evaluated potential is:

$$u(x) = \sum_m \left[ \frac{\partial G_k(x, y_m)}{\partial n_m} p_m - G_k(x, y_m) q_m \right] w_m,$$

where  $(y_m)$ ,  $(n_m)$ , and  $(w_m)$  come from the cached source quadrature data.

SPL is reported as:

$$\text{SPL}(x) = 20 \log_{10} \left( \frac{|u(x)|}{20 \mu\text{Pa}} \right).$$

Horizontal, vertical, and spherical observation points are concatenated into one field evaluation per frequency, then sliced back into result arrays. This avoids rebuilding source strengths for

each observation set.

## Performance Shape

The expensive stages are:

- dense operator assembly, roughly ( $O(N_e^2 q^2)$ ), where ( $N_e$ ) is face count and ( $q$ ) is quadrature point count
- dense direct solve, roughly ( $O(N_p^3)$ ), where ( $N_p$ ) is P1 dof count
- field evaluation, roughly ( $O(N_{\mathrm{obs}} N_e q)$ )

Symmetry can improve runtime by more than the simple physical-area reduction would suggest because it reduces the dense system dimension. Assembly has to include image contributions, so its scaling is closer to the expected half-domain or quarter-domain work plus image passes. The direct Burton-Miller solve, however, scales roughly as ( $O(N_p^3)$ ). Halving the P1 unknown count can make the dense solve approach one eighth of the full cost, and quartering it can make that portion much smaller still.

## Important Files

- `src/blab/solvers/beat_engine_backend.py`: Python adapter that stages assets and streams JSON events.
- `src/blab/solvers/julia_local/solver.jl`: request handling, mesh/radiator setup, frequency loop, backend dispatch, drive calculation.
- `src/blab/solvers/julia_local/src/BeatEngineCore.jl`: mesh representation, shared quadrature/formulation code, Burton-Miller solve, field evaluation interfaces.
- `src/blab/solvers/julia_local/src/BeatEngineCpu.jl`: include hub for the CPU implementation files.
- `src/blab/solvers/julia_local/src/BeatEngineCuda.jl`: include hub for the CUDA implementation files.

## BEAT Engine CPU

The BEAT Engine CPU backend is a hardware-agnostic Julia/OpenBLAS solver path. It uses the same BEAT Engine request protocol, mesh handling, Burton-Miller formulation, symmetry model, and result stream described in [BEAT Engine Core](#), but performs operator assembly, dense solve, and field evaluation on the host.

The application exposes this path as `BEAT Engine (CPU) / beat_cpu`.

## CPU Solve Path

For each frequency, the CPU path:

1. Selects the regular quadrature rule for that frequency.

2. Assembles dense Galerkin operators on host matrices.
3. Applies singular and image-singular Duffy corrections directly into those matrices.
4. Builds the Burton-Miller dense system.
5. Solves through Julia's dense BLAS/LAPACK path.
6. Evaluates requested polar and spherical fields on the CPU.

The CPU implementation lives under `src/blab/solvers/julia_local/src/BeatEngineCpu*.jl`.

## CPU Operator Assembly

`BeatEngineCpuAssembly.jl` is the CPU Galerkin operator assembly entry point. It precomputes per-element geometry and per-element quadrature data, then loops over test/trial element pairs.

Regular pair assembly skips adjacent/coincident pairs, which are handled afterward by singular corrections. When threaded assembly is enabled and Julia has more than one thread, element coloring is used to avoid write conflicts while scattering element-block contributions into shared dense matrices.

Symmetry image contributions are assembled on the host by reflecting trial/source element geometry and quadrature data. Reflected image pairs that become singular across a symmetry plane use image-singular correction caches.

## Dynamic Quadrature

The CPU path defaults to wavelength-driven regular quadrature. This is a CPU-only production feature; CUDA currently remains fixed-order as the GPU solver does not meaningfully benefit from dynamic quadrature.

The goal is to reduce low-frequency regular-pair assembly cost without changing singular-pair quadrature. Regular-pair assembly dominates dense BEM workload, and low frequencies do not need the same regular quadrature density as high frequencies on typical Boundary Lab loudspeaker meshes.

The selector computes:

$$kh = \frac{2\pi f}{c} \sqrt{A_{\text{stat}}},$$

where ( $A_{\text{stat}}$ ) is a mesh element-area statistic. The current default statistic is `p90`, so ( $h = \sqrt{A_{\text{p90}}}$ ).

Default CPU thresholds:

- q1 disabled: `wavelength_kh_q1_max = 0.0`
- q2 when  $k \cdot h \leq 2.0$

- q4 above  $k \cdot h > 2.0$
- base order: `quadrature_order = 4`
- mesh statistic: `wavelength_mesh_stat = "p90"`

The selected order is cached per frequency order. The CPU path reuses per-order regular rules, identity/mass matrices, and field-evaluation caches. Singular quadrature remains controlled by `singular_order` and is not reduced by the wavelength selector.

Result diagnostics include the selected mode, selected order, mesh statistic, element length, and  $k \cdot h$  value:

- `regular_quadrature_mode`
- `regular_quadrature_order`
- `regular_quadrature_base_order`
- `regular_quadrature_wavelength_mesh_stat`
- `regular_quadrature_wavelength_element_length_m`
- `regular_quadrature_wavelength_kh`
- `regular_quadrature_wavelength_kh_q1_max`
- `regular_quadrature_wavelength_kh_q2_max`

## Validation Notes

The tuned defaults were validated against fixed q4 on `sample.msh` using output-level checks from `compare_cpu_quadrature.jl`.

The strongest current candidate was:

- q1 disabled
- q2 while  $k \cdot h \leq 2.0$
- q4 above that
- p90 mesh-area statistic

On `sample.msh`, this selected q2 through 4 kHz and q4 at 4.5 kHz and above. The full-mesh output comparison measured approximately:

- median operator assembly speedup: 3.26x
- max pressure relative L2 error: 0.148%
- max field relative L2 error: 0.442%
- max SPL RMS delta: 0.116 dB
- max SPL p95 delta: 0.265 dB
- max SPL max delta: 0.356 dB

Raw operator relative error can exceed 1% even when solved pressure and field outputs remain well below the intended output-error target. Threshold tuning should therefore prioritize solved/output metrics over operator norms alone.

## CPU Field Evaluation

`BeatEngineCpuField.jl` evaluates the same field integral described in [BEAT Engine Core](#). It uses the CPU field-evaluation cache for the selected quadrature order and computes the potential at concatenated horizontal, vertical, and optional spherical observation points.

Field evaluation is usually not the dominant CPU cost compared with dense operator assembly and dense solve.

## CPU Dense Solve

`BeatEngineCpuSolve.jl` forms the Burton-Miller system on host matrices and calls Julia's dense solve path. Runtime depends heavily on BLAS/LAPACK performance and P1 unknown count. Symmetry can reduce solve cost significantly because dense solve complexity scales roughly as  $O(N_p^3)$ .

## CPU Benchmark And Comparison Scripts

Useful scripts:

- `src/blab/solvers/julia_local/scripts/benchmark_cpu.jl`: CPU timing benchmark, including fixed and wavelength regular quadrature modes.
- `src/blab/solvers/julia_local/scripts/compare_cpu_quadrature.jl`: fixed-reference versus candidate comparison artifact generator with operator, pressure, field, and SPL error metrics.

Example comparison:

```
& 'C:\Users\John\AppData\Local\Programs\Julia-1.12.6\bin\julia.exe' `
src\blab\solvers\julia_local\scripts\compare_cpu_quadrature.jl `
--frequencies 20,50,100,200,500,1000,1500,2000,3000,4000,4500,5000 `
--subset-faces 0 `
--output-points 72 `
--wavelength-kh-q1-max 0 `
--wavelength-kh-q2-max 2.0 `
--json src\blab\solvers\julia_local\results\cpu_quadrature_compare_outp
```

## Important Files

- `src/blab/solvers/julia_local/src/BeatEngineCpu.jl`: include hub for the CPU implementation files.
- `src/blab/solvers/julia_local/src/BeatEngineCpuAssembly.jl`: CPU Galerkin operator assembly entry point.
- `src/blab/solvers/julia_local/src/BeatEngineCpuField.jl`: CPU field-

evaluation path.

- `src/blab/solvers/julia_local/src/BeatEngineCpuSolve.jl`: CPU Burton-Miller dense solve through Julia's LAPACK/BLAS path.
- `src/blab/solvers/julia_local/solver.jl`: CPU backend dispatch, wavelength quadrature selection, per-order caches, and result diagnostics.

## BEAT Engine CUDA

The BEAT Engine CUDA backend is an Nvidia GPU-accelerated Julia solver path. It uses the same BEAT Engine request protocol, mesh handling, Burton-Miller formulation, symmetry model, and result stream described in [BEAT Engine Core](#), but performs regular-pair assembly, singular corrections, dense solve, and field evaluation with CUDA.jl.

The application exposes this path as `BEAT Engine (CUDA) / beat_cuda`.

## CUDA Operator Assembly

The CUDA path assembles dense Galerkin operators by splitting triangle pairs into regular pairs and adjacent/coincident singular pairs. Regular pairs are assembled by CUDA kernels over test/trial element pairs. Adjacent, edge-sharing, vertex-sharing, and coincident pairs are handled by a separate GPU Duffy correction path.

The application backend requires CUDA and moves geometry and quadrature arrays to the GPU:

- face vertices
- normals
- areas
- global face indices
- surface curls
- quadrature points and weights
- test/trial element index arrays

`build_cuda_regular_assembly_cache` keeps these arrays resident across frequencies, avoiding repeated host-to-device transfers for fixed mesh geometry.

For each frequency, CUDA allocates real and imaginary dense matrices for:

- SLP real/imag
- DLP real/imag
- adjoint DLP real/imag
- hypersingular real/imag

The regular-pair kernel skips adjacent pairs. Singular and near-singular corrections use Duffy quadrature and are added after regular assembly.

## CUDA Singular Corrections

A Duffy block kernel computes compact per-pair correction blocks directly on device. A CUDA scatter kernel atomically accumulates those compact blocks into dense correction buffers before adding them to the resident operators. The Duffy kernel reuses the regular assembly geometry cache, so the per-frequency singular correction path does not transfer dense CPU correction matrices.

The CUDA singular correction cache stores:

- test and trial element indices
- rule indices for orientation-remapped Duffy rules
- P1 row/column dofs and DP0 columns
- pair Jacobian scales and normal products
- flattened remapped Duffy points and weights

Symmetry image-singular pairs use the same compact correction idea, with reflected image geometry and GPU scatter.

## CUDA Kernel Modes

The CUDA backend has a regular assembly split kernel, one singular correction block kernel, and two field-evaluation kernels.

Regular assembly uses the multipair balanced split path by default. Each regular test/trial element pair still gets a fixed thread subgroup, but a CUDA block now carries multiple independent element pairs. The production layout uses 16 threads per element pair and 8 element pairs per CUDA block, for 128 threads per block. Per-thread partial sums are reduced inside each pair subgroup in dynamic shared memory:

```
scratch = CUDA.@cuDynamicSharedMem(typeof(k), blockDim().x * accumulator_c
```

The multipair path uses two regular assembly launches:

- `_cuda_regular_quadrature_slp_hyp_kernel!` computes single-layer and hypersingular contributions.
- `_cuda_regular_quadrature_dlp_adjoint_kernel!` computes double-layer and adjoint double-layer contributions.

This grouping keeps both launches at 24 accumulator slots, which reduces register/shared-memory pressure compared to a fused all-operator kernel. The subgroup mapping keeps the 16-thread-per-pair granularity that works well for the current order-4 regular triangle rule while avoiding the old one-pair-per-block launch shape.

The multipair balanced split kernels atomically scatter real and imaginary element-block entries into dense operator buffers. Singular adjacent/coincident pairs are skipped during regular assembly and handled afterward by the Duffy correction path. The previous one-pair-per-block

balanced path remains available as `:split_atomic_balanced` for profiling and A/B comparison.

`_cuda_duffy_blocks_kernel!` maps GPU threads over cached adjacent/coincident element pairs. Each thread computes the compact singular correction block for one or more pairs using the cached remapped Duffy rule. `_cuda_singular_scatter_kernel!` then atomically scatters those compact blocks into dense GPU correction buffers.

`_cuda_weighted_field_sources_kernel!` maps GPU threads over cached source quadrature points and builds weighted pressure and Neumann source strengths.

`_cuda_field_eval_kernel!` maps one CUDA block to one observation point and reduces source contributions in dynamic shared memory.

## CUDA Atomics

The GPU kernels accumulate element-block contributions into global dense matrices. Multiple CUDA blocks can target the same global row/column entries, especially because P1 basis functions are shared across adjacent faces. The regular assembly and singular scatter paths use device atomics for global accumulation:

```
@inline function _cuda_atomic_add!(array, index, value)
    CUDA.@atomic array[index] += value
    return nothing
end
```

Dense operator accumulation buffers are stored as separate real and imaginary arrays during kernel execution, so atomic additions operate on scalar `Float32` values. After the kernel finishes, real and imaginary matrices are materialized into complex matrices on GPU:

$$A = A_{\text{re}} + iA_{\text{im}}.$$

This avoids a slow serial scatter stage and lets regular-pair assembly remain massively parallel. The multipair balanced split regular assembly path preserves this property: it atomically accumulates into the same dense buffers through two balanced operator-family kernels instead of one all-operator kernel. The tradeoff is that floating-point atomic accumulation is order-dependent, so tiny run-to-run differences can occur at the last few bits.

## GPU Dense Solve

If operators are returned on GPU, `solve_burton_miller_neumann` forms the Burton-Miller matrix and RHS directly as `CuArrays`:

$$A = \frac{1}{2}I - D + \eta H,$$
$$b = \left(-S - \eta(D^* + \frac{1}{2}I_{P1, DP0})\right)q.$$

It then calls Julia's dense linear solve:

```
d_pressure = d_lhs \ d_rhs
```

With CUDA arrays, this dispatches through CUDA.jl to GPU dense linear algebra. The solved pressure vector is currently copied back to CPU after the solve; CUDA field evaluation then transfers pressure and Neumann vectors back to the GPU for the observation pass. This keeps the public result path simple while leaving room to keep pressure resident across downstream stages later.

Temporary GPU allocations are explicitly released with `CUDA.unsafe_free!` after assembly or solve stages to reduce memory pressure during frequency sweeps.

## CUDA Field Evaluation

`build_cuda_field_evaluation_cache` mirrors the shared field-evaluation cache to GPU as a `CudaFieldEvaluationCache`, using structure-of-arrays storage for coalesced source-point and normal reads.

For each frequency, CUDA field evaluation uses two stages:

1. `_cuda_weighted_field_sources_kernel!` interpolates solved P1 pressure to every source quadrature point, multiplies by quadrature weights, and builds the weighted Neumann source term.
2. `_cuda_field_eval_kernel!` assigns one CUDA block to each observation point. Threads in the block stride over all source quadrature points, evaluate the single-layer and double-layer kernels, accumulate local real/imaginary potentials, and reduce those partial sums in dynamic shared memory.

The result is materialized as a compact potential vector and copied back for SPL conversion and result serialization.

## Quadrature Mode

CUDA currently uses fixed regular quadrature order for the frequency sweep. Wavelength-driven regular quadrature is implemented only for the BEAT CPU path at this time.

CUDA's regular assembly cache is built from `mesh + rule`, so future CUDA wavelength quadrature would need per-order CUDA regular caches and corresponding field/identity cache handling.

## Performance Notes

CUDA accelerates regular-pair assembly, singular Duffy corrections, the dense solve, and field evaluation. The remaining dominant cost in CUDA solves is usually regular-pair assembly for the dense operators, followed by the dense solve for larger P1 systems. The multipair balanced split regular assembly mode reduces regular-kernel pressure by assembling SLP/hypersingular and DLP/adjoint in separate launches while keeping dense operators resident on the GPU and batching 8 regular pairs per CUDA block.

Temporary allocation, dense GPU memory footprint, and atomic accumulation cost are

important practical limits. CUDA memory use scales with dense P1/DP0 matrix dimensions, so symmetry is especially useful on large meshes.

`scripts/benchmark_cuda.jl` exposes the regular assembly mode for local comparison:

```
julia scripts\benchmark_cuda.jl --regular-assembly-mode multipair
julia scripts\benchmark_cuda.jl --regular-assembly-mode balanced
```

The production application path uses `multipair` by default. The helper scripts `scripts/profile_ncu_regular_multipair.ps1` and `scripts/profile_ncu_regular_balanced.ps1` run comparable Nsight Compute captures for the two regular split modes.

## Important Files

- `src/blab/solvers/julia_local/src/BeatEngineCuda.jl`: include hub for the CUDA implementation files.
- `src/blab/solvers/julia_local/src/BeatEngineCudaCommon.jl`: CUDA package setup, shared cache structs, and shared device helpers.
- `src/blab/solvers/julia_local/src/BeatEngineCudaRegular.jl`: CUDA geometry/rule cache builders and regular-pair kernels.
- `src/blab/solvers/julia_local/src/BeatEngineCudaSingular.jl`: GPU Duffy corrections, singular cache mirroring, image-singular corrections, and scatter kernels.
- `src/blab/solvers/julia_local/src/BeatEngineCudaOperators.jl`: GPU operator storage helpers, timing helpers, and regular kernel launch helpers.
- `src/blab/solvers/julia_local/src/BeatEngineCudaAssembly.jl`: public CUDA Galerkin operator assembly entry point.
- `src/blab/solvers/julia_local/src/BeatEngineCudaField.jl`: GPU field-evaluation cache, source weighting, and observation kernels.
- `src/blab/solvers/julia_local/src/BeatEngineCudaProfiling.jl`: optional CUDA regular-kernel probe and profiling launches used by benchmark scripts.

## Advanced CLI Workflow

Boundary Lab still includes command-line tools for mesh cleaning, solving, data preparation, and static plot generation. The GUI is the recommended entry point for normal use.

The CLI workflow is:

1. `blab clean`
2. `blab solve`
3. `blab prepare`
4. `blab plot`

## Clean A Mesh

```
blab clean input.msh output_clean.msh --merge-tol 1e-9
```

This merges coincident vertices, removes degenerate or duplicate triangles, and writes Gmsh 2.2 format.

## Run A Solve

```
blab solve output_clean.msh --output-npz pressure_data_raw.npz --freq-min
```

Useful options include:

- `--config`
- `--output-npz`
- `--freq-min`
- `--freq-max`
- `--freq-count`
- `--step-size`
- `--min-angle`
- `--max-angle`
- `--axial-offset`
- `--workers`
- `--gmres-tol`
- `--spherical-sampling`
- `--spherical-sampling-points`

For multi-mesh and multi-radiator TOML config files, see [solver-configuration.md](#).

## Prepare Visualization Data

```
blab prepare pressure_data_raw.npz pressure_data_formatted.npz --min-db -3
```

This applies clipping, interpolation, normalization, and smoothing for the static plot pipeline.

## Generate Static Plots

```
blab plot pressure_data_formatted.npz --output-dir .
```

This writes:

- `horizontal_isobar.png`
- `vertical_isobar.png`
- `acoustic_impedance.png`

## Notes

The CLI path is useful for scripted workflows and regression checks. It does not expose the full GUI project workflow or live plot update behavior.